Agentic Harness in Practice

2026-10-03

Table of contents

Building Blocks 2
Bootstrapping the Harness 5
Here Be Dragons 8
References 8
Bibliography 8

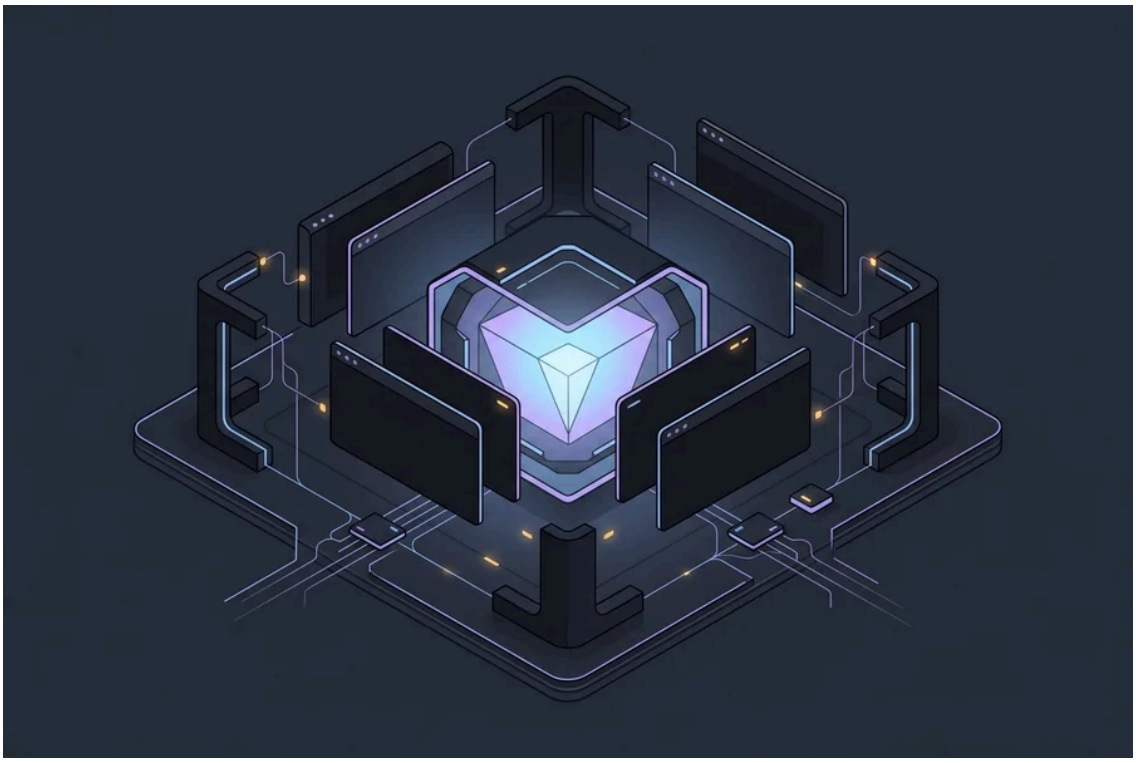


Figure 1: Agentic Harness cover

Ask a fresh coding agent to add a swap pricer to a quant codebase. Thirty seconds later you get clean, plausible code. It stores notionals as `float` and uses Pandas where your repository uses Polars. It passes curves around as bare dictionaries instead of your `YieldCurve` type. None of it is wrong in the abstract. All of it is wrong for your system.

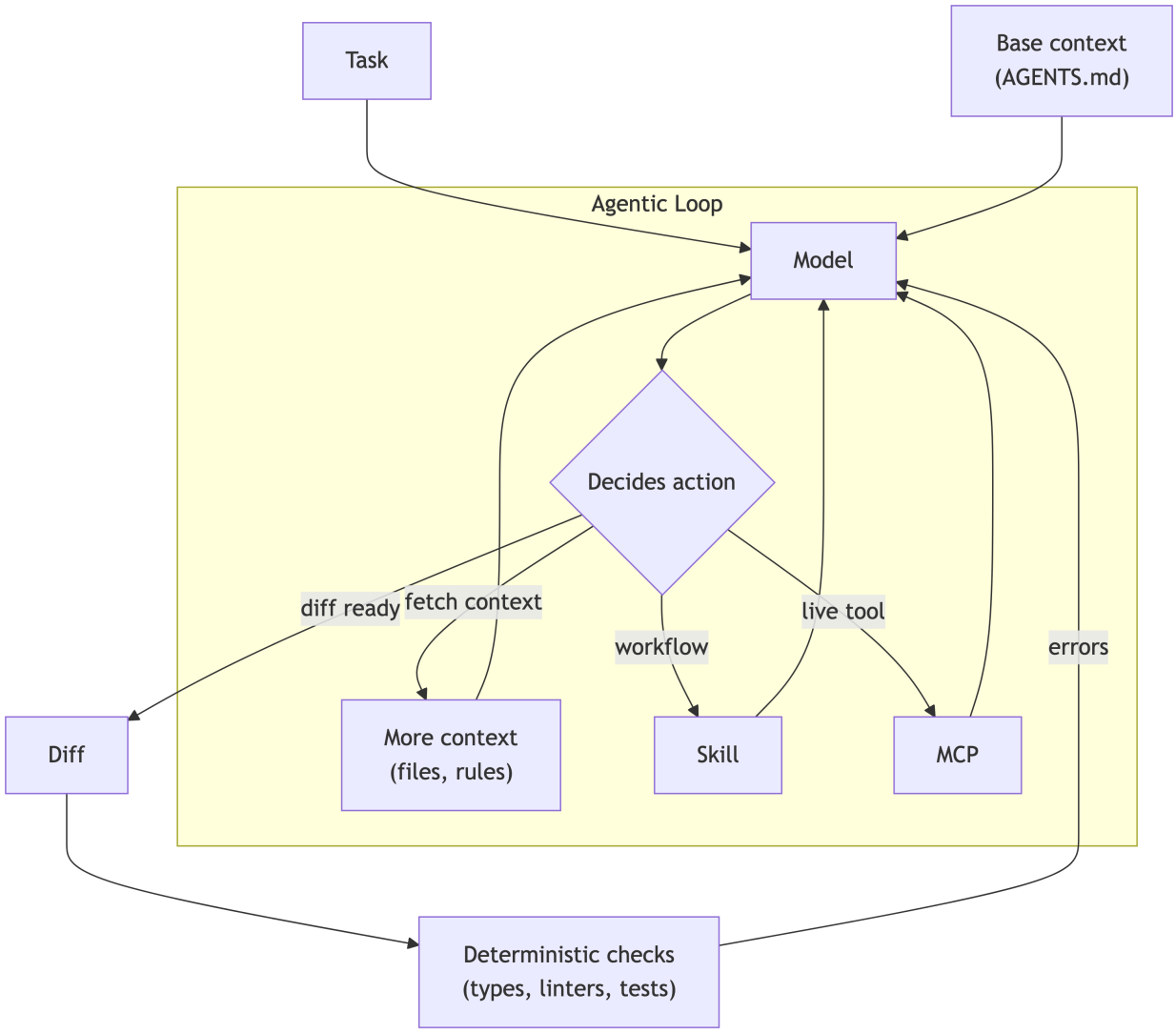
The model is not the problem. The harness is. Every AI coding tool ships with one: Cursor, Claude Code, and Antigravity each supply a default system prompt and a standard tool set. Those defaults

are generic. They contain no context about your architecture, business requirements, deprecated libraries, or the edge cases you spent weeks solving.

Operating with default settings costs real time and tokens. The model drifts, introduces subtle regressions, and burns context tokens on repetitive corrections. An agent is not just the model. It is the model plus its harness [1], [2], [3]. Build that missing harness directly into your repository, and an unpredictable model becomes a reliable engineering tool.

Building Blocks

The harness orchestrates two loops around the model. In the inner loop, the model fetches more context, loads skills, or queries MCP tools to produce a diff. In the outer loop, deterministic checks catch failures and feed errors back for correction.



Context

Context represents the general instructions, project goals, and business domain you provide to the model.

Keep your context small. Context is your most expensive asset. Every token in your base prompt consumes working memory and incurs compounding cost on every single turn [4].

Divide your context into two scopes:

1. **Global scope**: Project purpose, core practices, and non-negotiable boundaries. This belongs in a root `AGENTS.md` file [5].
2. **Local scope**: Directory-specific conventions and module rules. Harnesses load local rules only for matching files. Examples include a nested `AGENTS.md` in a subdirectory, or a Cursor rule matching `src/pricing/**/*.py`.

Skills

Skills represent on-demand capabilities. Unlike global context, skills do not load into memory all at once. They package workflows into modular `SKILL.md` files following the open skill format [6].

When a session begins, the harness exposes only skill frontmatter: its name and description. That metadata costs roughly fifty tokens per skill. The runtime injects full instructions into context only when a prompt matches that specific workflow.

Because routing depends on this frontmatter, the `description` must explicitly state both what the skill does and **when to run it**. Frame descriptions with imperative trigger conditions (*"Use when..."*) and user intent rather than internal mechanics. Clear triggers prevent false negatives and ensure reliable activation.

Here is a minimal skill example:

```
---
name: verify-release
description: >
  Run the pre-release checklist and verify distribution artifacts. Use when
   tagging a release, preparing a PyPI publication, or validating wheels
before
  deployment.
---

# Release Verification Workflow

1. Run the test suite: `uv run pytest`
2. Build the wheel: `uv run pyproject-build`
3. Inspect package metadata: `uv run twine check dist/*`
```

Skills come in several distinct flavors across the ecosystem:

- **Output formatting and token efficiency**: Enforcing communication contracts and reducing output waste. Examples include `caveman` to strip verbose boilerplate, ASD-STE100 to enforce unambiguous Simplified Technical English, and Conventional Commits to produce structured git histories.
- **Architectural discipline and methodology**: Guiding how the model approaches problem-solving. `ponytail` forces the model to verify standard library alternatives before adding dependencies. Other skills enforce test-driven development (TDD) via red-green-refactor cycles or mandate spec-plan-execute reviews before touching code.
- **Risk assessment and pre-flight triage**: Surfacing edge cases and architectural hazards before execution. Before migrations, I run Shreyas Doshi's pre-mortem rubric. It sorts threats into tigers (real risks), paper tigers (false alarms), and elephants (unspoken blockers).
- **Performance profiling and debugging**: Automating diagnostic routines for memory and CPU bottlenecks. A profiling skill can instruct the model to run Memray for memory allocation tracking or execute py-spy to sample running processes.
- **Operational automation and release lifecycles**: Wrapping domain-specific tools and distribution sequences into verifiable scripts. Examples include running Alembic schema migration checks, orchestrating Twine package verification, or crawling site routes with broken-link checkers.

Model Context Protocol (MCP)

Where markdown skills define procedural workflows, the **Model Context Protocol (MCP)** standardizes live tool execution [7]. MCP servers allow the harness to query external systems—such as schema inspectors, issue trackers, or documentation indices—on demand without bloating base context tokens.

Pairing MCP with Retrieval-Augmented Generation (RAG) gives the agent targeted memory. Instead of dumping entire documentation archives or historical pull requests into your base prompt, an internal RAG pipeline indexes reference materials and returns only the relevant passages on query. MCP provides the standardized runtime bridge: the model invokes a retrieval tool, inspects the retrieved context, and continues execution without degrading prompt cache efficiency.

Deterministic Tooling

Many developers rely on long prompt instructions to enforce coding standards. That is a mistake. Probabilistic models drift. Deterministic tools evaluate explicit rules.

Know the modern tooling ecosystem for your stack, and use it aggressively. In Python, that means full type annotation coverage combined with a fast static type checker like `pyrefly`. Models struggle on real-world repository tasks without reproducible environments and tight feedback loops [8].

Consider this riddle: What does this code do?

```
1   def prepare_objects(data, preparer):
2       objects = []
```

```
    for d in data:
        o = preparer(d)
        objects.append(o)
    return objects
```

What shape is `data`? What input type does `preparer` accept? What does it return? Neither a human developer nor a type checker can answer without inspecting caller implementations.

Now consider the typed equivalent. The function body is identical. Only the annotations changed:

```
import datetime as dt
from collections.abc import Callable

type YieldCurve = Callable[[float], float]
type ParSwapQuote = tuple[dt.date, float]


def prepare_objects(
    data: list[list[ParSwapQuote]],
    preparer: Callable[[list[ParSwapQuote]], YieldCurve],
) -> list[YieldCurve]:
    objects: list[YieldCurve] = []
    for d in data:
        o = preparer(d)
        objects.append(o)
    return objects
```

Line 4

Domain type aliases: a curve maps tenor (in years) to rate, a quote is a maturity date and a par rate.

Line 9

One list of quotes per curve to build.

Line 10

The callable contract states exactly what goes in and what comes out.

This version requires more effort to write. Yet it is much more readable. That clarity helps human engineers, and it cuts generation errors further. If generated code passes a single quote where the contract expects a list, or reads a missing attribute, `pyrefly` rejects it immediately.

Bootstrapping the Harness

Your mileage may vary, but this is the exact sequence I follow when setting up a project.

The key technique is **bootstrapping**. Use the model itself to generate the initial harness configuration.

Tooling comes first because every later step leans on it: `AGENTS.md` mandates the check command, and skills execute it. Once you set that foundation, the model helps expand the harness by scaffolding custom skills and writing bespoke verification scripts. Those new tools wire straight back into the primary check command, compounding the harness over time.

1. Tooling First

Deterministic tooling is your most reliable investment and offers the highest ROI. Fiddling with system prompts without automated assertions is an exercise in futility [9]. You need concrete gates that catch regressions instantly.

Configuring linters, formatters, and type checkers by hand can take time. Do not do it manually. Use pre-made templates, or define desired standards and prompt the model to generate the configuration.

Create a single script command (like `poe code-quality` or `make check`) that runs everything. Then wire it in three places:

- **The model's own loop**: The model runs the command after every change, reads the errors, and fixes them before it reports done. This is the cheapest feedback you will ever get. It costs seconds and no human attention.
- **Pre-commit hooks**: Catch what slipped through before it lands in history.
- **CI pipelines**: The final gate that nobody can skip.

2. Context

Next, write a minimal `AGENTS.md`. Mine rarely exceeds twenty lines:

```
# AGENTS.md

## Project
Pricing library for interest-rate swaps. Python 3.13, managed with `uv`.

## Commands
- Install: `uv sync`
- Check everything: `uv run poe code-quality`
- Tests only: `uv run pytest -q`

## Rules
- Run `uv run poe code-quality` after every change. Fix all errors before you
  report done.
- Use `Decimal` for money. Never `float`.
- Start each session: read the target module, `git status`, and the relevant
tests.
```

```
16
17  ## Do not touch
18  - `vendor/` (generated code)
19  - `migrations/` (hand-reviewed only)
```

Two elements matter most: your project goal, and the mandate that all code must pass the deterministic quality checks from step 1. Never cram twenty formatting rules into `AGENTS.md`. Let linters and type checkers enforce mechanical standards.

3. Spec-Driven Development

Establish your own practices for specification.

A model generates incorrect implementations when requirements are vague. Before requesting code generation, define three elements:

- **Intent**: The problem you are solving.
- **Constraints**: What cannot break.
- **Acceptance criteria**: The tests that prove success.

Resolving ambiguity in a short specification takes minutes. Fixing a 500-line pull request built on incorrect assumptions takes hours.

4. Published Skills

The ecosystem now offers thousands of community skills, installable with a single `npx skills add` command.

A new hype wave appears every week. Approach published skills with care:

- **Security risks**: Community skills often contain executable scripts and shell commands. Connecting models to private files and active tools creates severe prompt-injection risks [10]. Always audit the underlying code before running them.
- **Skill bloat**: Overloading the environment with 100 skills degrades tool selection. Routing accuracy drops sharply as tool counts grow and descriptions overlap [11].

Experiment, evaluate what works, and keep your skill inventory lean.

5. Custom Skills

Most modern agentic tools include skill-authoring capabilities in their default harness. Use them.

When you type the same prompt instructions three times, turn that workflow into a skill. Document the procedure, prompt the model to scaffold the skill file, verify the steps, and commit the file to version control.

Pay close attention to the description field. List concrete trigger contexts (*"Use when migrating a database schema or writing Alembic revisions"*) so the model activates the file reliably.

Treat custom skills as version-controlled code. Review modifications to them through pull requests just as you would application code.

6. Bespoke Tooling

Build your own bespoke tooling. You understand the unique character of your project better than anyone else.

Ask yourself these questions:

- What hidden conventions do experienced team members follow?
- What architectural invariants must we guarantee?
- Where do repeated mistakes happen?

Spend half a day identifying those pain points. Then prompt the model to write a custom lint script or verification tool that detects violations. Wire that tool directly into your local check command.

Here Be Dragons

A healthy dose of realism is necessary when working with agentic systems:

- **Developer skills decay**: Outsourcing all debugging and implementation dulls technical intuition [12]. You must understand generated code deeply enough to catch subtle algorithmic failures or silent memory leaks during review.
- **The "solved software" myth**: Marketing claims that prompt engineering solves software production. The fantasy of a fully autonomous "dark factory" where code writes itself without human intervention remains an illusion [13]. Real engineering remains hard.
- **Token costs are real**: Multi-turn agent loops burn tokens fast. Every turn re-sends your base context (see Context), and automated loops that cycle through the same failing test multiply that cost. Most harnesses report per-session token usage. Look at it weekly, and trim whatever context the model never needed.
- **Review fatigue and rubber-stamping**: Automated tools generate hundreds of plausible lines in seconds. The bottleneck shifts from writing code to auditing it. Large-scale code quality data reveals marked increases in code churn and duplicate logic when review fatigue sets in [14]. If tired engineers rubber-stamp massive diffs, subtle technical debt rapidly pollutes the repository.
- **The "set it and forget it" trap**: A repository harness is not a static setup. As codebases evolve, libraries update, and model capabilities shift, your rules, skills, and deterministic gates require continuous maintenance.

If you do one thing after reading this, make it the single check command from step 1. Wire it into the model's loop today. Everything else in this post builds on it.

References

Bibliography

[1] A. Osmani, "Agent Harness Engineering." [Online]. Available: https://addyosmani.com/blog/agent-harness-engineering/

[2] A. Graziano, "How to set up your repository for AI coding agents." [Online]. Available: https://ainativesoftware.engineering/harness

[3] M. Hashimoto, "My AI Adoption Journey." [Online]. Available: https://mitchellh.com/writing/my-ai-adoption-journey

[4] M. Pound and Computerphile, "Why AI Tokens are so Expensive." [Online]. Available: https://www.youtube.com/watch?v=-0HRzXk8vlk

[5] GitHub Next, "AGENTS.md: An open convention for AI coding agents." [Online]. Available: https://agents.md/

[6] Agent Skills, "Agent Skills Format Specification." [Online]. Available: https://agentskills.io/specification

[7] Anthropic, "Model Context Protocol (MCP) Specification." [Online]. Available: https://modelcontextprotocol.io/

[8] C. E. Jimenez *et al.*, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.06770

[9] H. Husain, "Your AI Product Needs Evals." [Online]. Available: https://hamel.dev/blog/posts/evals/

[10] S. Willison, "The Lethal Trifecta of AI Agent Security and Prompt Injection." [Online]. Available: https://simonwillison.net/2025/Jun/18/the-lethal-trifecta/

[11] E. Yan, "Patterns for Building LLM-Based Systems & Products." [Online]. Available: https://eugeneyan.com/writing/llm-patterns/

[12] A. Osmani, "Agentic Skill Decay." [Online]. Available: https://addyosmani.com/blog/agentic-skill-decay/

[13] A. Osmani, "Software Factories, Light and Dark." [Online]. Available: https://addyosmani.com/blog/software-factories/

[14] GitClear, "Coding on Copilot: 2023 Data Suggests Downward Pressure on Code Quality," 2024. [Online]. Available: https://www.gitclear.com/coding_on_copilot_data_shows_downward_pressure_on_code_quality